

Approach -

Goal – To find best driver and increase number of accepted orders by participants (drivers).
Currently using Binary Classification to get highest scores (probabilities) driver – wise, with the target column indicating if the participant accepted the order.

Alternative Approaches – A variety of methods can be tried to address the problem at hand (Additional data needed).

- Have the target column as ‘Participant Rating’ - Predict the rating of the drivers (regression) – return the top-rated drivers sorted by closest distance from pickup. Data for ‘Participant Rating’ can be collected from external sources or calculated from features like participant status, number of completed trips in the past etc.
- Have a mechanism to evaluate reason behind cancellation/ignorance by participants – take action to reflect this in their rating or other similar metrics. Provide additional incentives to participants in case of high demand to increase their acceptance, and ultimately reduce waiting time for customers.

Due to lack of time, I could not explore these possibilities, but would like to have similar discussions during interviews.

Implementation/Observations -

make_dataset.py

- The join of the booking and participant data does not make much sense if only one column is used for joining.

event_timestamp_x	order_id	booking_status	customer_id	driver_id_x	trip_distance	pickup_latitude	pickup_longitude	event_timestamp_y	driver_id_y	participant_status	experiment_key	driver_latitude	driver_longitude
2015-05-12 05:25:23.904 UTC	102254181112	CREATED	715281995014	NaN	2.0	-2.998461	104.788077	2015-05-12 05:25:24.056 UTC	221172732473	CREATED	697155500625	-2.998517	104.788077
2015-05-12 05:25:23.904 UTC	102254181112	CREATED	715281995014	NaN	2.0	-2.998461	104.788077	2015-05-12 05:25:24.548 UTC	221172732473	ACCEPTED	697155500625	-2.998517	104.788077
2015-05-12 05:25:24.67 UTC	102254181112	DRIVER_FOUND	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.056 UTC	221172732473	CREATED	697155500625	-2.998517	104.788077
2015-05-12 05:25:24.67 UTC	102254181112	DRIVER_FOUND	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.548 UTC	221172732473	ACCEPTED	697155500625	-2.998517	104.788077
2015-05-12 05:55:10.496544 UTC	102254181112	COMPLETED	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.056 UTC	221172732473	CREATED	697155500625	-2.998517	104.788077

I am not fully sure, but as per my understanding the join doesn't make sense as there are multiple records for the same 'order_id' in both booking and participant data, and join would result in $n*m$ records, where the values of 'participant_status' and 'booking_status' together won't make sense for a particular record.

participant_log - Excel

Ganisha Chaturvedi

File Home Insert Page Layout Formulas Data Review View Help Tell me

D47 412665252012

	B	C	D	E
1	driver_id	participant_status	order_id	experiment_key
47	5.33035E+11	IGNORED	4.12665E+11	9.22214E+11
51	5.33035E+11	CREATED	4.12665E+11	9.22214E+11
93895	8.40364E+11	ACCEPTED	4.12665E+11	9.22214E+11
93896	8.40364E+11	CREATED	4.12665E+11	9.22214E+11

booking_log - Excel

Ganisha Chaturvedi

File Home Insert Page Layout Formulas Data Review View Help Tell me

412665252012

	A	B	C
1	event_timestamp	order_id	booking_status
365063	2015-05-12 07:01:01.283 UTC	4.12665E+11	CREATED
365064	2015-05-12 07:01:18.706 UTC	4.12665E+11	DRIVER_FOUND
365065	2015-05-12 07:03:30.282 UTC	4.12665E+11	PICKED_UP
365066	2015-05-12 07:14:32.668 UTC	4.12665E+11	COMPLETED

- Joining using multiple columns ('order_id','driver_id') might yield a more meaningful result.

event_timestamp_x	order_id	booking_status	customer_id	driver_id	trip_distance	pickup_latitude	pickup_longitude	event_timestamp_y	participant_status	experiment_key	driver_latitude	driver_longitude
2015-05-12 05:25:24.67 UTC	102254181112	DRIVER_FOUND	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.056 UTC	CREATED	697155500625	-2.998517	104.780779
2015-05-12 05:25:24.67 UTC	102254181112	DRIVER_FOUND	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.548 UTC	ACCEPTED	697155500625	-2.998517	104.780779
2015-05-12 05:55:10.496544 UTC	102254181112	COMPLETED	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.056 UTC	CREATED	697155500625	-2.998517	104.780779
2015-05-12 05:55:10.496544 UTC	102254181112	COMPLETED	715281995014	2.211727e+11	2.0	-2.998461	104.788077	2015-05-12 05:25:24.548 UTC	ACCEPTED	697155500625	-2.998517	104.780779
2015-05-12 05:29:38.214 UTC	105520285203	DRIVER_FOUND	957980952237	5.276724e+11	2.0	-2.970621	104.756226	2015-05-12 05:29:38.123 UTC	ACCEPTED	243283239235	-2.970765	104.750961

- When a new request is created, the record is first added to booking_log then to participant_log.
- Duplicates present in both participant and booking files are removed. Initially, I was unable to understand why the key is just 4 columns while removing duplicates, therefore edited that. Also, couldn't experiment between left and inner join, so proceeded with inner join.

build_features.py and transformations.py

- Booking data is used to calculate number of completed bookings driver-wise, as 'participant_status' in participant_log does not have 'completed' value.
- `df.drop(columns=['event_timestamp_participant','customer_id','experiment_key','event_timestamp_booking','pickup_latitude','pickup_longitude','driver_latitude','driver_longitude','driver_id_booking'], inplace=True)`
`df = pd.get_dummies(df, columns=['booking_status', 'participant_status'], dtype=int)`
 reason for dropping columns - event timestamp - hour extracted, ID columns – not needed, however, during model training only features mentioned in config.toml are used so this step is optional, latitude and longitude details removed as distance is calculated.
- One-hot encoded categorical features 'booking_status' and 'participant_status'.
- Tried training model on data which has encoded categorical features, but code was failing while predicting drivers for test_data file as the schema is different. Joining 'test_data' with 'booking_log' and 'participant_log' gives no results on 'order_id'. Experimented few other ways to get test_data aligned with one-hot encoded version of training data (eg - join by driver_id), but could not complete the process.
- The performance metrics on below dataset (feature engg + categorical encoding) were all '1' (100%) – which could be explored further.

```
transf_df_v2.head()
```

compl_count	order_id	trip_distance	driver_id_participant	driver_gps_accuracy	is_completed	driver_distance	event_hour	booking_status_COMPLETED	booking_status_CUSTOMI
1	865687867130	1.37	100049799902	68.4	1	0.097266	3	0	
1	865687867130	1.37	100049799902	68.4	0	0.097266	3	0	
1	865687867130	1.37	100049799902	68.4	1	0.097266	2	0	
1	865687867130	1.37	100049799902	68.4	0	0.097266	2	0	
1	865687867130	1.37	100049799902	68.4	1	0.097266	3	1	

classifier.py train_model.py and predict_model.py

- Accuracy, precision, recall and f1-score are used, other metrics can also be explored.
- Hyperparameter tuning can be added – I changed n_estimators = 300 to 100, max_depth=30 to 10, bootstrap=false to true and test_size=0.2 to 0.3.
- Feature engineering applied to 'test_data' to make it compatible with the model for predictions.

Other Observations/Scope for Improvements:

- The target column is created from 'accepted' value of participant_status field, but the column name ultimately is 'is_completed', 'accepted' cannot be used to indicate completion as the participant can cancel even after accepting, this could be explored further.
- The join of participant and booking data can be made more meaningful.
- Class imbalance should be checked.
- Checked data discrepancy – while removing duplicates, if other columns like 'customer_id' have unique values where set of these 4 columns (order_id, trip_distance, pickup_latitude, pickup_longitude) are duplicate – that would indicate data discrepancy – no discrepancy found.
- Booking data doesn't give information when a driver ignored a request, and after a driver 'ignores' – new 'created' status is not added when looking for another driver.